

Documentación Técnica — DevSecOps

Documentación de referencia del diseño, las decisiones técnicas y el funcionamiento del proyecto.

Contenido

1. [DevSecOps y shift-left security](#)
2. [SAST — análisis estático](#)
3. [SCA — composición de software](#)
4. [DAST — pruebas dinámicas](#)
5. [Gating por severidad](#)
6. [Endurecimiento de la aplicación](#)
7. [OWASP ZAP como escáner completo](#)
8. [Vías de extensión](#)
9. [Glosario](#)

1. DevSecOps y shift-left security

La seguridad tratada como una etapa final —una auditoría antes de salir a producción— llega tarde: los defectos ya están profundamente integrados y arreglarlos es caro. **DevSecOps** integra la seguridad al ciclo de desarrollo, y **shift-left** significa moverla lo más temprano posible: a cada commit y cada pull request.

Este proyecto materializa ese principio con tres capas de análisis automático que corren en el pipeline, cada una cubriendo un ángulo distinto que las otras no ven.

2. SAST — análisis estático

Static Application Security Testing analiza el **código fuente** sin ejecutarlo, buscando patrones inseguros. Se usa Semgrep con un conjunto de reglas locales y deterministas (`security/semgrep-rules.yml`), que detectan:

- **Command injection:** `exec()` con entrada dinámica interpolada.
- **SQL injection:** queries construidas por concatenación/interpolación de strings.
- **Uso de `eval`:** ejecución de código arbitrario.
- **Secretos hardcodedos:** credenciales embebidas en el código.

El gate corre Semgrep sobre `src/` con `--error`, que hace fallar el build si hay hallazgos. Como la aplicación está escrita de forma segura, el gate pasa. Para demostrar que las reglas **sí** detectan, se corren sobre `security/examples-insecure/` —snippets vulnerables a propósito, que no forman parte de la app— donde producen los cuatro hallazgos esperados.

Usar reglas locales (en lugar del registry en la nube) hace el análisis **determinista y offline**: los mismos hallazgos en cada corrida, sin depender de una descarga externa.

3. SCA — composición de software

Software Composition Analysis analiza las **dependencias** en busca de vulnerabilidades conocidas (CVEs). La mayor parte del código de una aplicación moderna son librerías de terceros; una de ellas con un CVE grave es una puerta de entrada, sin que tu propio código tenga un solo error.

El gate (`scripts/sca-gate.mjs`) corre `npm audit`, parsea el reporte y **falla si hay vulnerabilidades de severidad high o critical**. Las de nivel moderate/low se reportan pero no bloquean (política configurable).

Caso real de este proyecto: durante el desarrollo, el gate detectó una vulnerabilidad *high/critical* real —el advisory de `esbuild` (GHSA-67mh-4wv8-2f99), una dependencia transitiva del runner de tests— y bloqueó. Se remedió actualizando la dependencia a una versión parcheada, tras lo cual el gate quedó en verde. Esto es exactamente para lo que sirve el SCA: atrapar el problema en el pipeline, no en producción.

4. DAST — pruebas dinámicas

Dynamic Application Security Testing analiza la aplicación **en ejecución**, atacándola como lo haría un adversario y observando las respuestas. A diferencia del SAST, ve el comportamiento real en runtime.

Este proyecto cubre la capa dinámica con **pruebas de seguridad dirigidas** (`tests/security.test.ts`): levantan la app real y verifican propiedades de seguridad concretas:

- **Headers de seguridad:** presencia de `X-Content-Type-Options`, `X-Frame-Options`, `Content-Security-Policy`, `Strict-Transport-Security`, y ausencia de `X-Powered-By`.
- **Resistencia a XSS reflejado:** un payload `<script>` se devuelve escapado, no crudo.
- **Resistencia a inyección en login:** un payload tipo `' OR '1'='1` no autentica, y el mensaje de error no revela si el usuario existe.
- **Control de acceso:** el recurso protegido responde 401 sin token, 403 con token inválido y 200 con token válido.

Escribir pruebas de seguridad dirigidas complementa a un escáner genérico: el escáner encuentra clases conocidas de problemas; las pruebas propias verifican las reglas de seguridad específicas del dominio de la aplicación.

5. Gating por severidad

La clave de un pipeline de seguridad útil es que **bloquee lo grave sin ahogar en ruido**. La política aplicada:

- **SAST**: cualquier hallazgo de las reglas (severidad ERROR) bloquea.
- **SCA**: vulnerabilidades *high* o *critical* bloquean; *moderate/low* se reportan.
- **DAST**: las pruebas dinámicas son asserts binarios; cualquier fallo bloquea.

El principio general: un gate demasiado laxo no protege; uno demasiado estricto (que bloquea por cada hallazgo trivial) termina siendo desactivado por el equipo. La calibración por severidad es lo que hace el gate sostenible.

6. Endurecimiento de la aplicación

La app (`src/server.ts`) está escrita para resistir las vulnerabilidades del OWASP Top 10 más comunes:

- **XSS (A03)**: todo dato reflejado en HTML se escapa (`escapeHtml`).
- **Inyección (A03)**: el login usa coincidencia exacta y comparación en tiempo constante (`timingSafeEqual`), sin construir queries por concatenación.
- **Control de acceso roto (A01)**: el recurso protegido exige un token válido, distinguiendo 401 (no autenticado) de 403 (sin permiso).
- **Configuración de seguridad (A05)**: headers de seguridad en todas las respuestas; no se expone la tecnología del servidor (`X-Powered-By`); los errores no filtran stack traces.
- **Filtración de información**: el login devuelve el mismo mensaje para usuario inexistente y password incorrecta, sin revelar cuál falló.

7. OWASP ZAP como escáner completo

Las pruebas dinámicas dirigidas verifican reglas específicas; un **escáner DAST completo** como OWASP ZAP complementa rastreando la aplicación y probando un catálogo amplio de vulnerabilidades conocidas de forma pasiva (baseline) o activa.

El workflow `.github/workflows/zap-dast.yml` provee un escaneo baseline de ZAP como referencia, disparable manualmente: levanta la app y corre `zapproxy/action-baseline`, publicando el reporte. En una política estricta, los hallazgos de nivel FAIL de ZAP también actuarían como gate. Se mantiene separado del pipeline principal para que este sea determinista, mientras el escaneo completo queda disponible bajo demanda.

8. Vías de extensión

- **SAST con reglas del registry:** complementar las reglas locales con conjuntos curados (`p/security-audit` , `p/owasp-top-ten`).
- **Escaneo de secretos dedicado:** integrar gitleaks/trufflehog para el historial completo, además del patrón de SAST.
- **SCA con SBOM:** generar un *Software Bill of Materials* (CycloneDX) y verificarlo contra bases de vulnerabilidades.
- **DAST activo:** pasar de baseline (pasivo) a un escaneo activo de ZAP en un entorno de staging.
- **laC scanning:** analizar configuraciones de infraestructura (Docker, Terraform) con herramientas como Checkov.
- **Gate de política unificado:** consolidar los hallazgos de todas las capas en formato SARIF y aplicar una política única.

9. Glosario

- **DevSecOps:** integración de la seguridad en el ciclo de desarrollo y el pipeline.
- **Shift-left security:** mover los controles de seguridad lo más temprano posible.
- **SAST:** análisis estático del código fuente en busca de patrones inseguros.
- **SCA:** análisis de las dependencias en busca de vulnerabilidades conocidas.
- **DAST:** análisis dinámico de la aplicación en ejecución.
- **CVE:** identificador público de una vulnerabilidad conocida.
- **Quality gate:** verificación automática que el código debe pasar para avanzar.
- **OWASP Top 10:** lista de referencia de los riesgos de seguridad web más críticos.
- **XSS:** inyección de scripts que se ejecutan en el navegador de otro usuario.
- **Command/SQL injection:** inyección de comandos o SQL por falta de sanitización.
- **Baseline scan:** escaneo pasivo (sin atacar activamente) que reporta debilidades observables.
- **SBOM:** inventario de todos los componentes de software de una aplicación.